

Overloaded Constructor

Quiz 02 — Instructor key (omit from print run)

Question **model answer**

Add an overloaded constructor for `Stack` that takes the initial capacity as a parameter, in place of the hard-coded `new int[10]`, so that `Stack S1(100);` compiles and gives `S1` room for 100 elements.

// WRITTEN DIRECTLY BELOW THE GIVEN STACK()

```
Stack(int capacity) {
    array = new int[capacity];
    top = -1;
}
```

GRADING NOTES**What counts**

Min

One `int` parameter (any name — `capacity`, `size`, `n` all fine) used as the `new[]` length, and `top = -1;` still set. Parameter order/name is not graded.

Watch — unused param

`Stack(int capacity) { array = new int[10]; top = -1; }` — parameter declared but never used, so `Stack S1(100)` still only gets 10 slots. This is the most common miss; no credit for the array line, since it defeats the point of overloading.

Watch — missing top

Forgetting `top = -1;` leaves `top` uninitialized garbage, so the first `push ( array[ ++top] )` writes to a garbage index. Half credit if the array line is otherwise correct.

Bonus, not required

Storing the capacity in a member (e.g. `int capacity;`, set here) so later code can bounds-check `push`. The prompt only asked for the constructor, so its absence should not be penalized.

Auto-deductions

Rewriting `push`, adding a destructor, or changing the given `Stack()` — the prompt asked only for the new constructor to be added below it.

Heads up (not a student issue)

The given class declares `Stack()` before the `public:` label, so it (and, if placed literally “below it” per the prompt, the overload too) is technically `private` — meaning `Stack S;` / `Stack S1(100);` in `main()` wouldn't actually compile as printed. Don't dock students for this; it's a pre-existing issue with the printed class, not something the prompt asked them to notice or fix. A student who places both constructors under `public:` instead has, if anything, given a *more* correct answer.